Indian Institute of Technology Delhi

Department of Artificial Intelligence

Deep Learning

Assignment-2

Submitted By: Tarun Kumar
Roll Number: 2025JRB2023

Submitted To: Prof. James Abaram
Course: Deep Learning (AIL 7021)

Academic Year 2025–2026

→ Bird Species Classification

1 Introduction

This project focuses on multi-class bird species classification with 15 categories. The objective was to design and train a deep learning model from scratch without using pretrained weights.

2 Data Preprocessing

2.1 Dataset Split

The dataset was split using stratified sampling:

- 80% Training data
- 20% Validation data

Stratification preserved class distribution.

2.2 Handling Class Imbalance

Class weights were computed using inverse frequency:

$$w_i = \frac{1}{n_i}$$

The weights were normalized and passed to the loss function.

2.3 Data Augmentation

To reduce overfitting:

- Random Horizontal Flip
- Random Rotation (±18°)
- Random Resized Crop (0.9–1.0 scale)
- Color Jitter (contrast adjustment)

All images were resized to 224×224.

3 Model Architecture

A custom ResNet-style CNN was implemented.

3.1 Architecture Details

- Initial 7×7 convolution (stride 2)
- Batch Normalization
- Max Pooling
- Four Residual Layers:
  - Layer 1: 64 channels
  - Layer 2: 128 channels
  - Layer 3: 256 channels
  - Layer 4: 512 channels
- Global Average Pooling
- Dropout (0.4)
- Fully Connected Layer (512 → 15)

3.2 Residual Block

Each residual block contains:

- Conv(3×3) + BN + ReLU
- Conv(3×3) + BN
- Skip Connection (Identity or 1×1 Conv)

Residual connections improve gradient flow and allow deeper networks.

4 Loss Function

CrossEntropyLoss was used with:

- Class weights
- Label smoothing = 0.1

Label smoothing formula:

$$y_{smooth} = (1 - \epsilon)y + \frac{\epsilon}{K}$$

Where:

- $\epsilon = 0.1$
- $K = 15$

5 Optimizer

Adam optimizer was used with:

- Learning Rate: 1×10^{-3}

Adam was chosen for faster convergence and adaptive learning rates.

6 Learning Rate Scheduler

ReduceLROnPlateau was used:

- Monitored: Final Score
- Factor: 0.5
- Patience: 5 epochs
- Minimum LR: 10^{-6}

7 Regularization Techniques

To prevent overfitting:

- Data augmentation
- Dropout (0.4)
- Label smoothing
- Class weighting
- Early stopping (patience = 10)

8 Evaluation Metrics

Performance was evaluated using:

- Accuracy
- Macro F1-score
- Micro F1-score

Final Score:

$$\text{Final Score} = \frac{\text{Macro F1} + \text{Micro F1}}{2}$$

Macro F1 balances minority classes, Micro F1 reflects overall accuracy.

9 Training Configuration

- Image Size: 224×224
- Batch Size: 64
- Epochs: 40
- Early Stopping Patience: 10
- Classes: 15

10 Results and Discussion

The model showed stable convergence. Learning rate scheduling improved performance when validation score plateaued.

Combining:

- Residual architecture
- Class-weighted loss
- Label smoothing
- Dropout

resulted in improved generalization.

11 Training Results and Performance Analysis

```
Epoch [1/40] Train Loss: 2.4997 | Train Acc: 0.2377 | Val Loss: 2.3200 | Val Acc: 0.3067 | Macro F1: 0.2761 | Micro F1: 0.3067 | Final Score: 0.2914
Epoch [2/40] Train Loss: 2.0444 | Train Acc: 0.4184 | Val Loss: 4.4629 | Val Acc: 0.1565 | Macro F1: 0.0928 | Micro F1: 0.1565 | Final Score: 0.1246
Epoch [3/40] Train Loss: 1.8505 | Train Acc: 0.4956 | Val Loss: 1.8758 | Val Acc: 0.4826 | Macro F1: 0.4819 | Micro F1: 0.4826 | Final Score: 0.4823
Epoch [4/40] Train Loss: 1.6904 | Train Acc: 0.5606 | Val Loss: 1.7394 | Val Acc: 0.5724 | Macro F1: 0.5629 | Micro F1: 0.5724 | Final Score: 0.5676
Epoch [5/40] Train Loss: 1.5565 | Train Acc: 0.6282 | Val Loss: 1.4412 | Val Acc: 0.6730 | Macro F1: 0.6608 | Micro F1: 0.6730 | Final Score: 0.6669
Epoch [6/40] Train Loss: 1.4340 | Train Acc: 0.6778 | Val Loss: 1.5515 | Val Acc: 0.6217 | Macro F1: 0.6272 | Micro F1: 0.6217 | Final Score: 0.6244
Epoch [7/40] Train Loss: 1.3175 | Train Acc: 0.7290 | Val Loss: 1.6835 | Val Acc: 0.5989 | Macro F1: 0.5805 | Micro F1: 0.5989 | Final Score: 0.5897
Epoch [8/40] Train Loss: 1.2207 | Train Acc: 0.7735 | Val Loss: 1.4774 | Val Acc: 0.6776 | Macro F1: 0.6638 | Micro F1: 0.6776 | Final Score: 0.6707
Epoch [9/40] Train Loss: 1.1621 | Train Acc: 0.7973 | Val Loss: 1.1254 | Val Acc: 0.8146 | Macro F1: 0.8074 | Micro F1: 0.8146 | Final Score: 0.8110
Epoch [10/40] Train Loss: 1.0960 | Train Acc: 0.8264 | Val Loss: 1.3288 | Val Acc: 0.7454 | Macro F1: 0.7325 | Micro F1: 0.7454 | Final Score: 0.7390
Epoch [11/40] Train Loss: 1.0271 | Train Acc: 0.8491 | Val Loss: 1.2397 | Val Acc: 0.7632 | Macro F1: 0.7454 | Micro F1: 0.7632 | Final Score: 0.7543
Epoch [12/40] Train Loss: 1.0019 | Train Acc: 0.8609 | Val Loss: 1.2066 | Val Acc: 0.7724 | Macro F1: 0.7792 | Micro F1: 0.7724 | Final Score: 0.7758
Epoch [13/40] Train Loss: 0.9438 | Train Acc: 0.8822 | Val Loss: 1.1885 | Val Acc: 0.7831 | Macro F1: 0.7768 | Micro F1: 0.7831 | Final Score: 0.7799
Epoch [14/40] Train Loss: 0.9264 | Train Acc: 0.8876 | Val Loss: 1.0656 | Val Acc: 0.8291 | Macro F1: 0.8342 | Micro F1: 0.8291 | Final Score: 0.8316
Epoch [15/40] Train Loss: 0.8880 | Train Acc: 0.9048 | Val Loss: 1.0704 | Val Acc: 0.8278 | Macro F1: 0.8211 | Micro F1: 0.8278 | Final Score: 0.8245
Epoch [16/40] Train Loss: 0.8664 | Train Acc: 0.9139 | Val Loss: 1.0352 | Val Acc: 0.8489 | Macro F1: 0.8433 | Micro F1: 0.8489 | Final Score: 0.8461
Epoch [17/40] Train Loss: 0.8536 | Train Acc: 0.9166 | Val Loss: 1.0051 | Val Acc: 0.8634 | Macro F1: 0.8551 | Micro F1: 0.8634 | Final Score: 0.8593
Epoch [18/40] Train Loss: 0.8170 | Train Acc: 0.9321 | Val Loss: 0.9010 | Val Acc: 0.9011 | Macro F1: 0.8987 | Micro F1: 0.9011 | Final Score: 0.8999
Epoch [19/40] Train Loss: 0.8030 | Train Acc: 0.9332 | Val Loss: 0.9692 | Val Acc: 0.8853 | Macro F1: 0.8803 | Micro F1: 0.8853 | Final Score: 0.8828
Epoch [20/40] Train Loss: 0.7989 | Train Acc: 0.9388 | Val Loss: 0.9429 | Val Acc: 0.8791 | Macro F1: 0.8767 | Micro F1: 0.8791 | Final Score: 0.8779
Epoch [21/40] Train Loss: 0.7743 | Train Acc: 0.9453 | Val Loss: 0.9659 | Val Acc: 0.8729 | Macro F1: 0.8707 | Micro F1: 0.8729 | Final Score: 0.8718
Epoch [22/40] Train Loss: 0.7641 | Train Acc: 0.9532 | Val Loss: 0.8928 | Val Acc: 0.8986 | Macro F1: 0.8953 | Micro F1: 0.8986 | Final Score: 0.8969
Epoch [23/40] Train Loss: 0.7401 | Train Acc: 0.9588 | Val Loss: 0.8923 | Val Acc: 0.8945 | Macro F1: 0.8925 | Micro F1: 0.8945 | Final Score: 0.8935
Epoch [24/40] Train Loss: 0.7375 | Train Acc: 0.9591 | Val Loss: 1.2151 | Val Acc: 0.7703 | Macro F1: 0.7715 | Micro F1: 0.7703 | Final Score: 0.7709
Epoch [25/40] Train Loss: 0.6818 | Train Acc: 0.9818 | Val Loss: 0.7682 | Val Acc: 0.9454 | Macro F1: 0.9436 | Micro F1: 0.9454 | Final Score: 0.9445
Epoch [26/40] Train Loss: 0.6610 | Train Acc: 0.9887 | Val Loss: 0.7456 | Val Acc: 0.9528 | Macro F1: 0.9500 | Micro F1: 0.9528 | Final Score: 0.9514
Epoch [27/40] Train Loss: 0.6604 | Train Acc: 0.9882 | Val Loss: 0.7357 | Val Acc: 0.9565 | Macro F1: 0.9538 | Micro F1: 0.9565 | Final Score: 0.9552
Epoch [28/40] Train Loss: 0.6507 | Train Acc: 0.9913 | Val Loss: 0.7543 | Val Acc: 0.9458 | Macro F1: 0.9433 | Micro F1: 0.9458 | Final Score: 0.9446
Epoch [29/40] Train Loss: 0.6489 | Train Acc: 0.9922 | Val Loss: 0.7399 | Val Acc: 0.9512 | Macro F1: 0.9487 | Micro F1: 0.9512 | Final Score: 0.9499
Epoch [30/40] Train Loss: 0.6439 | Train Acc: 0.9920 | Val Loss: 0.7603 | Val Acc: 0.9478 | Macro F1: 0.9438 | Micro F1: 0.9478 | Final Score: 0.9458
Epoch [31/40] Train Loss: 0.6457 | Train Acc: 0.9910 | Val Loss: 0.7473 | Val Acc: 0.9495 | Macro F1: 0.9469 | Micro F1: 0.9495 | Final Score: 0.9482
Epoch [32/40] Train Loss: 0.6424 | Train Acc: 0.9933 | Val Loss: 0.7350 | Val Acc: 0.9512 | Macro F1: 0.9489 | Micro F1: 0.9512 | Final Score: 0.9500
Epoch [33/40] Train Loss: 0.6351 | Train Acc: 0.9952 | Val Loss: 0.7274 | Val Acc: 0.9528 | Macro F1: 0.9514 | Micro F1: 0.9528 | Final Score: 0.9521
Epoch [34/40] Train Loss: 0.6254 | Train Acc: 0.9969 | Val Loss: 0.7144 | Val Acc: 0.9623 | Macro F1: 0.9600 | Micro F1: 0.9623 | Final Score: 0.9612
Epoch [35/40] Train Loss: 0.6195 | Train Acc: 0.9980 | Val Loss: 0.7186 | Val Acc: 0.9594 | Macro F1: 0.9585 | Micro F1: 0.9594 | Final Score: 0.9590
Epoch [36/40] Train Loss: 0.6185 | Train Acc: 0.9981 | Val Loss: 0.7138 | Val Acc: 0.9594 | Macro F1: 0.9579 | Micro F1: 0.9594 | Final Score: 0.9586
Epoch [37/40] Train Loss: 0.6171 | Train Acc: 0.9990 | Val Loss: 0.7099 | Val Acc: 0.9586 | Macro F1: 0.9571 | Micro F1: 0.9586 | Final Score: 0.9578
Epoch [38/40] Train Loss: 0.6165 | Train Acc: 0.9989 | Val Loss: 0.7123 | Val Acc: 0.9611 | Macro F1: 0.9589 | Micro F1: 0.9611 | Final Score: 0.9600
Epoch [39/40] Train Loss: 0.6144 | Train Acc: 0.9992 | Val Loss: 0.7054 | Val Acc: 0.9640 | Macro F1: 0.9619 | Micro F1: 0.9640 | Final Score: 0.9630
Epoch [40/40] Train Loss: 0.6145 | Train Acc: 0.9988 | Val Loss: 0.7077 | Val Acc: 0.9615 | Macro F1: 0.9608 | Micro F1: 0.9615 | Final Score: 0.9612
```

Figure 1: Training log showing loss, accuracy, Macro F1, Micro F1, and Final Score across 40 epochs.

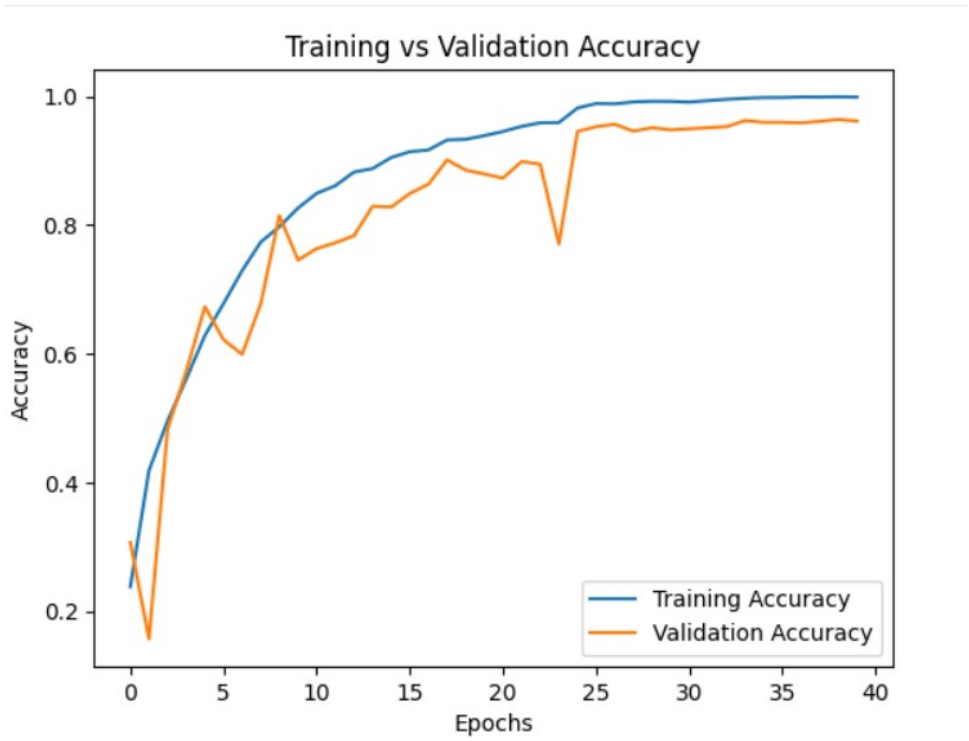

Figure 2: Training vs Validation Accuracy over epochs.

12 Conclusion

A custom ResNet-based CNN was successfully implemented from scratch. Careful hyperparameter tuning and regularization techniques significantly improved performance.

Future work may include:

- Cosine Annealing scheduler
- Mixup augmentation
- Deeper residual variants

→ Cell Segmentation Classification

1 Introduction

This project focuses on solving a multi-class image segmentation problem for cell images using a deep learning approach. The goal is to assign a class label to each pixel in the image. The segmentation model is trained using a U-Net architecture implemented in PyTorch.

The dataset contains RGB images and corresponding segmentation masks representing five classes including the background.

2 Data Preprocessing

All images and masks were resized to 320×320 pixels to maintain consistent input dimensions. Images were normalized to the range $[0, 1]$ by dividing pixel values by 255.

Segmentation masks originally stored as RGB images were converted into integer label maps using a predefined color-to-class mapping.

3 Data Augmentation

To improve model generalization and reduce overfitting, several data augmentation techniques were applied during training:

- Horizontal flipping
- Vertical flipping
- Random rotation
- Random brightness adjustment
- Gaussian noise
- Random zoom
- Gaussian blur

These augmentations increase dataset diversity and help the model learn robust features.

4 Architecture Choice

The segmentation model is based on the U-Net architecture, which is widely used for biomedical image segmentation.

U-Net consists of two main parts:

- Encoder (contracting path)
- Decoder (expanding path)

The encoder extracts hierarchical features using convolution and max pooling layers, while the decoder gradually reconstructs the spatial resolution using transposed convolutions.

Skip connections between encoder and decoder layers allow the model to combine low-level spatial features with high-level semantic information, improving segmentation accuracy.

The model uses four encoder blocks followed by a bottleneck layer and four decoder blocks.

5 Regularization Techniques

Several regularization methods were used to prevent overfitting:

- Dropout layers with probabilities of 0.2 and 0.3
- Batch normalization after convolution layers
- Weight decay (L2 regularization) in the optimizer
- Data augmentation

Dropout randomly disables neurons during training to improve generalization.

6 Loss Function

A combined loss function was used to improve segmentation performance, especially for imbalanced classes.

The final loss is defined as:

$$Loss = CE + Focal + 2 \times Dice \tag{1}$$

6.1 Cross Entropy Loss

Cross entropy measures pixel-wise classification error.

6.2 Focal Loss

Focal loss focuses training on difficult samples and reduces the impact of well-classified pixels.

6.3 Dice Loss

Dice loss measures overlap between predicted segmentation and ground truth.

$$Dice = \frac{2|X \cap Y|}{|X| + |Y|} \tag{2}$$

Dice loss is particularly effective for segmentation tasks with class imbalance.

7 Class Weighting

Class weights were computed based on pixel frequency in the dataset. The background class weight was reduced while the foreground classes were increased to balance the training process.

Logarithmic scaling was applied to stabilize the weights.

8 Hyperparameters

The model was trained using the following hyperparameters:

- Image size: 320×320
- Number of classes: 5
- Batch size: 8
- Learning rate: 1×10^{-4}
- Epochs: 120
- Weight decay: 1×10^{-4}

9 Optimizer

The Adam optimizer was used due to its adaptive learning rate capability and stable convergence properties.

$$\theta_{t+1} = \theta_t - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon} \tag{3}$$

A learning rate scheduler (ReduceLROnPlateau) was applied to decrease the learning rate when validation loss stops improving.

10 Training Strategy

The dataset was split into training and validation sets using an 80-20 split.

Early stopping was implemented with a patience of 10 epochs to prevent overfitting. The model with the lowest validation loss was saved during training.

11 Evaluation Metrics

Model performance was evaluated using the following metrics:

11.1 Intersection over Union (IoU)

$$IoU = \frac{TP}{TP + FP + FN} \tag{4}$$

11.2 Dice Score

$$Dice = \frac{2TP}{2TP + FP + FN} \tag{5}$$

The final mean IoU and Dice scores were calculated excluding the background class.

12 Training Results and Performance Analysis

```
Epoch [1/120] Train Loss: 3.7951 Val Loss: 3.9236 Train Acc: 0.4392 Val Acc: 0.7805
Epoch [2/120] Train Loss: 3.6115 Val Loss: 3.8162 Train Acc: 0.5372 Val Acc: 0.7797
Epoch [3/120] Train Loss: 3.5391 Val Loss: 3.5957 Train Acc: 0.5858 Val Acc: 0.7721
Epoch [4/120] Train Loss: 3.5135 Val Loss: 3.3952 Train Acc: 0.6089 Val Acc: 0.7174
Epoch [5/120] Train Loss: 3.3897 Val Loss: 3.3152 Train Acc: 0.6548 Val Acc: 0.6479
Epoch [6/120] Train Loss: 3.3354 Val Loss: 3.2922 Train Acc: 0.6611 Val Acc: 0.6408
Epoch [7/120] Train Loss: 3.2914 Val Loss: 3.1461 Train Acc: 0.6891 Val Acc: 0.6866
Epoch [8/120] Train Loss: 3.2915 Val Loss: 3.1837 Train Acc: 0.6881 Val Acc: 0.6611
Epoch [9/120] Train Loss: 3.2302 Val Loss: 3.1369 Train Acc: 0.7118 Val Acc: 0.6744
Epoch [10/120] Train Loss: 3.1742 Val Loss: 3.1020 Train Acc: 0.7276 Val Acc: 0.6804
Epoch [11/120] Train Loss: 3.1847 Val Loss: 3.0700 Train Acc: 0.7354 Val Acc: 0.6832
Epoch [12/120] Train Loss: 3.1181 Val Loss: 3.0643 Train Acc: 0.7377 Val Acc: 0.6962
Epoch [13/120] Train Loss: 3.1033 Val Loss: 3.0623 Train Acc: 0.7324 Val Acc: 0.6839
Epoch [14/120] Train Loss: 3.0983 Val Loss: 3.0267 Train Acc: 0.7500 Val Acc: 0.6972
Epoch [15/120] Train Loss: 3.0822 Val Loss: 3.0155 Train Acc: 0.7650 Val Acc: 0.6945
Epoch [16/120] Train Loss: 3.0460 Val Loss: 2.9956 Train Acc: 0.7685 Val Acc: 0.6970
Epoch [17/120] Train Loss: 3.0045 Val Loss: 3.0419 Train Acc: 0.7735 Val Acc: 0.7009
Epoch [18/120] Train Loss: 3.0342 Val Loss: 2.9148 Train Acc: 0.7749 Val Acc: 0.7403
Epoch [19/120] Train Loss: 3.0272 Val Loss: 2.9282 Train Acc: 0.7629 Val Acc: 0.7153
Epoch [20/120] Train Loss: 2.9543 Val Loss: 3.0129 Train Acc: 0.7896 Val Acc: 0.6981
Epoch [21/120] Train Loss: 2.9354 Val Loss: 2.9244 Train Acc: 0.7973 Val Acc: 0.7100
Epoch [22/120] Train Loss: 2.9425 Val Loss: 2.9715 Train Acc: 0.7946 Val Acc: 0.7146
Epoch [23/120] Train Loss: 2.9842 Val Loss: 2.8709 Train Acc: 0.7798 Val Acc: 0.7428
Epoch [24/120] Train Loss: 2.9177 Val Loss: 2.8666 Train Acc: 0.8051 Val Acc: 0.7568
Epoch [25/120] Train Loss: 2.8937 Val Loss: 2.8597 Train Acc: 0.8073 Val Acc: 0.7465
Epoch [26/120] Train Loss: 2.8651 Val Loss: 2.8982 Train Acc: 0.8064 Val Acc: 0.7170
Epoch [27/120] Train Loss: 2.8605 Val Loss: 2.8963 Train Acc: 0.8082 Val Acc: 0.7334
Epoch [28/120] Train Loss: 2.8274 Val Loss: 2.8012 Train Acc: 0.8207 Val Acc: 0.7620
Epoch [29/120] Train Loss: 2.8475 Val Loss: 2.8297 Train Acc: 0.8110 Val Acc: 0.7571
Epoch [30/120] Train Loss: 2.8052 Val Loss: 2.7557 Train Acc: 0.8231 Val Acc: 0.7768
Epoch [31/120] Train Loss: 2.8412 Val Loss: 2.7921 Train Acc: 0.8067 Val Acc: 0.7454
Epoch [32/120] Train Loss: 2.7895 Val Loss: 2.8743 Train Acc: 0.8247 Val Acc: 0.7234
Epoch [33/120] Train Loss: 2.8418 Val Loss: 2.7700 Train Acc: 0.8223 Val Acc: 0.7481
Epoch [34/120] Train Loss: 2.7833 Val Loss: 2.7853 Train Acc: 0.8095 Val Acc: 0.7354
Epoch [35/120] Train Loss: 2.7542 Val Loss: 2.7570 Train Acc: 0.8292 Val Acc: 0.7633
Epoch [36/120] Train Loss: 2.7907 Val Loss: 2.7687 Train Acc: 0.8222 Val Acc: 0.7562
Epoch [37/120] Train Loss: 2.7467 Val Loss: 2.7719 Train Acc: 0.8293 Val Acc: 0.7385
Epoch [38/120] Train Loss: 2.7336 Val Loss: 2.7369 Train Acc: 0.8348 Val Acc: 0.7732
Epoch [39/120] Train Loss: 2.7252 Val Loss: 2.7147 Train Acc: 0.8396 Val Acc: 0.7915
Epoch [40/120] Train Loss: 2.7136 Val Loss: 2.8355 Train Acc: 0.8394 Val Acc: 0.7290
Epoch [41/120] Train Loss: 2.7550 Val Loss: 2.6869 Train Acc: 0.8371 Val Acc: 0.7822
Epoch [42/120] Train Loss: 2.7301 Val Loss: 2.7183 Train Acc: 0.8425 Val Acc: 0.7790
Epoch [43/120] Train Loss: 2.7289 Val Loss: 2.6963 Train Acc: 0.8366 Val Acc: 0.7638
Epoch [44/120] Train Loss: 2.7310 Val Loss: 2.6994 Train Acc: 0.8458 Val Acc: 0.7745
Epoch [45/120] Train Loss: 2.7092 Val Loss: 2.6735 Train Acc: 0.8496 Val Acc: 0.7985
Epoch [46/120] Train Loss: 2.7027 Val Loss: 2.6819 Train Acc: 0.8497 Val Acc: 0.7824
Epoch [47/120] Train Loss: 2.6837 Val Loss: 2.7296 Train Acc: 0.8448 Val Acc: 0.7606
Epoch [48/120] Train Loss: 2.7107 Val Loss: 2.6978 Train Acc: 0.8385 Val Acc: 0.7666
Epoch [49/120] Train Loss: 2.6701 Val Loss: 2.6221 Train Acc: 0.8511 Val Acc: 0.7960
Epoch [50/120] Train Loss: 2.6733 Val Loss: 2.6109 Train Acc: 0.8454 Val Acc: 0.7950
Epoch [51/120] Train Loss: 2.7081 Val Loss: 2.7280 Train Acc: 0.8405 Val Acc: 0.7597
Epoch [52/120] Train Loss: 2.6764 Val Loss: 2.7161 Train Acc: 0.8447 Val Acc: 0.7668
Epoch [53/120] Train Loss: 2.6849 Val Loss: 2.6151 Train Acc: 0.8508 Val Acc: 0.7950
Epoch [54/120] Train Loss: 2.6311 Val Loss: 2.7225 Train Acc: 0.8523 Val Acc: 0.7548
Epoch [55/120] Train Loss: 2.6684 Val Loss: 2.6392 Train Acc: 0.8464 Val Acc: 0.7776
Epoch [56/120] Train Loss: 2.6253 Val Loss: 2.6867 Train Acc: 0.8485 Val Acc: 0.7601
Epoch [57/120] Train Loss: 2.6618 Val Loss: 2.6418 Train Acc: 0.8482 Val Acc: 0.7710
Epoch [58/120] Train Loss: 2.6766 Val Loss: 2.6215 Train Acc: 0.8466 Val Acc: 0.7845
Epoch [59/120] Train Loss: 2.6114 Val Loss: 2.6465 Train Acc: 0.8541 Val Acc: 0.7804
Epoch [60/120] Train Loss: 2.6183 Val Loss: 2.5800 Train Acc: 0.8551 Val Acc: 0.8035
Epoch [61/120] Train Loss: 2.5892 Val Loss: 2.5804 Train Acc: 0.8555 Val Acc: 0.8034
Epoch [62/120] Train Loss: 2.6111 Val Loss: 2.6073 Train Acc: 0.8506 Val Acc: 0.7924
Epoch [63/120] Train Loss: 2.6249 Val Loss: 2.6081 Train Acc: 0.8613 Val Acc: 0.7931
Epoch [64/120] Train Loss: 2.6468 Val Loss: 2.5759 Train Acc: 0.8519 Val Acc: 0.8048
Epoch [65/120] Train Loss: 2.6154 Val Loss: 2.5657 Train Acc: 0.8517 Val Acc: 0.8132
Epoch [66/120] Train Loss: 2.5988 Val Loss: 2.5791 Train Acc: 0.8609 Val Acc: 0.8060
Epoch [67/120] Train Loss: 2.6197 Val Loss: 2.5917 Train Acc: 0.8523 Val Acc: 0.7945
Epoch [68/120] Train Loss: 2.6257 Val Loss: 2.5628 Train Acc: 0.8458 Val Acc: 0.8004
Epoch [69/120] Train Loss: 2.5918 Val Loss: 2.5543 Train Acc: 0.8595 Val Acc: 0.8101
Epoch [70/120] Train Loss: 2.5906 Val Loss: 2.6118 Train Acc: 0.8641 Val Acc: 0.7929
Epoch [71/120] Train Loss: 2.5740 Val Loss: 2.6139 Train Acc: 0.8633 Val Acc: 0.7868
Epoch [72/120] Train Loss: 2.5590 Val Loss: 2.6109 Train Acc: 0.8599 Val Acc: 0.7885
Epoch [73/120] Train Loss: 2.5873 Val Loss: 2.6300 Train Acc: 0.8655 Val Acc: 0.7879
Epoch [74/120] Train Loss: 2.6246 Val Loss: 2.6364 Train Acc: 0.8514 Val Acc: 0.7791
Epoch [75/120] Train Loss: 2.5834 Val Loss: 2.6451 Train Acc: 0.8599 Val Acc: 0.7818
Epoch [76/120] Train Loss: 2.5545 Val Loss: 2.5887 Train Acc: 0.8627 Val Acc: 0.7933
Epoch [77/120] Train Loss: 2.5910 Val Loss: 2.5618 Train Acc: 0.8537 Val Acc: 0.7993
Epoch [78/120] Train Loss: 2.5816 Val Loss: 2.5796 Train Acc: 0.8661 Val Acc: 0.7940
Epoch [79/120] Train Loss: 2.5834 Val Loss: 2.5691 Train Acc: 0.8589 Val Acc: 0.7983
Early stopping triggered
```

Figure 3: Training vs Validation Accuracy over epochs 120.

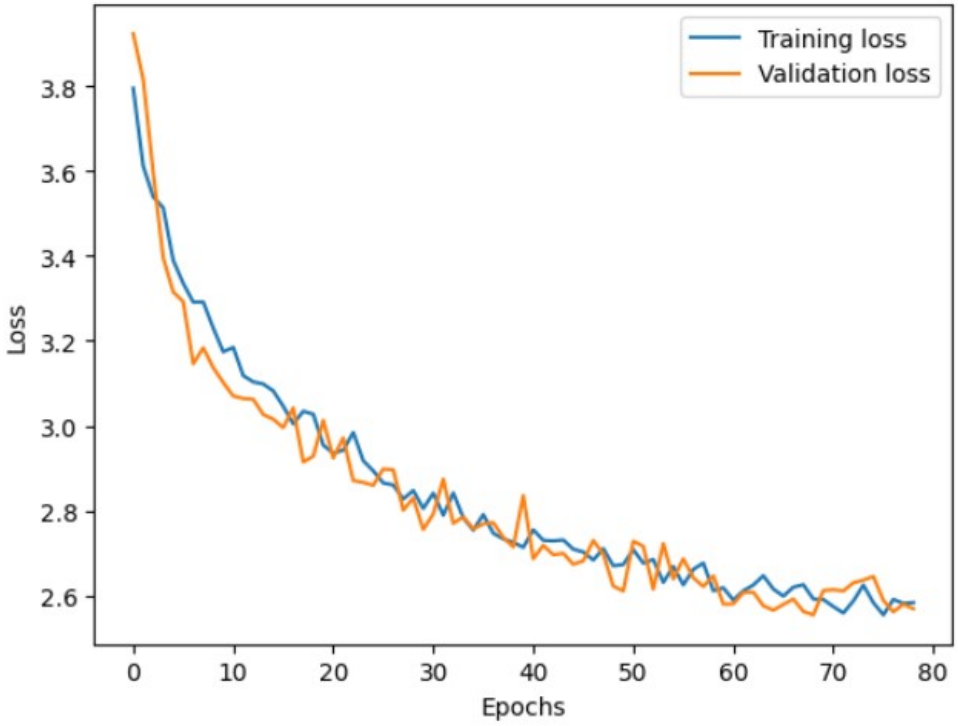

Figure 4: Training log showing loss across 120 epochs.

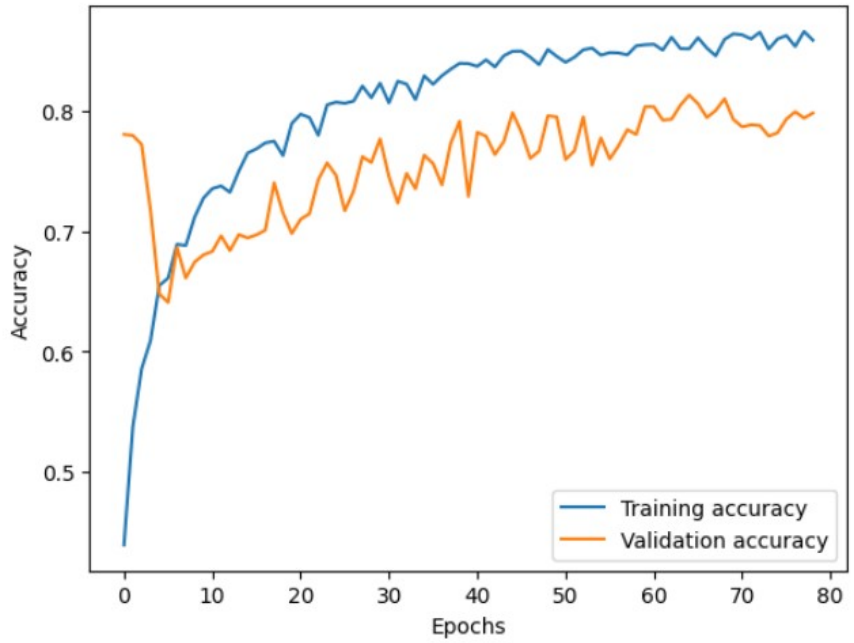

Figure 5: Training accuracy showing loss across 120 epochs.

13 Conclusion

The U-Net architecture combined with data augmentation, class weighting, and a hybrid loss function provides strong performance for multi-class cell segmentation. The use of Dice loss and Focal loss helps address class imbalance and improves segmentation quality.

Model Download Links

Bird and Cell model files can be downloaded from the following Google Drive folder:
Google Drive Model Folder